

The Cost of Centralised Leadership in Distributed Consensus

An Experimental Comparison of Leader-Based and Leaderless Consensus

Sephy Ryu
Independent Researcher
South Korea

Abstract

Keywords

consensus, Raft, EPaxos, leader-based ordering, leaderless ordering, benchmarking

1 Introduction

Distributed systems often replicate the same state across multiple machines in order to provide fault tolerance and continued service in the presence of failures. When a new operation modifies the state of a replicated state machine, replicas must agree on how that operation is ordered and applied. If different replicas apply operations in different orders, their states can diverge, violating the consistency guarantees expected from the system. Consensus is therefore a fundamental problem in distributed systems.

Paxos is one of the foundational consensus algorithms that established a formal approach to reaching agreement among distributed nodes in the presence of failures [2]. Despite its theoretical importance, Paxos is notoriously difficult to understand and implement. The original presentation is highly abstract and leaves substantial implementation details to system designers. As a result, practical systems based on Paxos may differ considerably in their architecture and implementation choices.

Raft was introduced with a different design goal: making consensus easier to understand and implement while maintaining strong fault-tolerance guarantees [4]. A central feature of Raft is its explicit leader. Client requests that modify replicated state are handled by the leader, which determines the log order and replicates entries to follower replicas before they are committed. This structure substantially simplifies the coordination problem. Instead of requiring every replica to participate symmetrically in ordering every operation, the system can rely on a clearly defined leader to coordinate the replication process.

This simplicity, however, introduces a structural trade-off. Because the leader is responsible for coordinating replicated writes, write processing becomes concentrated around a single replica. Increasing the number of replicas does not therefore provide a corresponding increase in the number of nodes directly responsible for ordering writes. The leader may consequently become a bottleneck as workload intensity increases. Furthermore, when the leader fails, the system must perform leader election before normal leader-based processing can resume. During this transition, request processing may be temporarily affected.

This raises an important question: how much does the centralisation of ordering actually cost?

An alternative approach is provided by leaderless consensus. Rather than assigning a single replica the responsibility of coordinating all operations, leaderless protocols allow multiple replicas to participate directly in determining the ordering and dependencies of operations. EPaxos is a representative example of this approach [3]. It exploits the independence of non-conflicting operations so that replicas can sometimes commit operations without relying on a single global leader.

The absence of a central leader, however, does not eliminate the costs of consensus. Leaderless protocols generally require additional mechanisms for identifying conflicts, tracking dependencies, and coordinating the execution of operations. These mechanisms can introduce additional communication and computational overhead. Consequently, leaderless consensus should not be assumed to outperform leader-based consensus in every workload. Its benefits may instead depend on factors such as operation conflicts, workload composition, network conditions, and failure patterns.

This creates a fundamental trade-off between two different approaches to distributed ordering. Leader-based consensus centralises ordering to simplify coordination, potentially reducing the complexity and resource cost of normal operation at the expense of concentrating write coordination and introducing dependence on leader availability. Leaderless consensus distributes the ordering process across replicas, potentially reducing dependence on a single ordering point while introducing additional coordination overhead.

We experimentally compare Raft and EPaxos as representative leader-based and leaderless consensus protocols. The objective is not to determine that one consensus model is universally superior, but to examine the practical costs and benefits of their different ordering structures under controlled workloads. The evaluation considers both normal operation and failure scenarios, including read- and write-intensive workloads, leader failures, and replica failures.

We focus on the following research questions:

- **RQ1:** How do leader-based and leaderless consensus differ in steady-state throughput and latency across read/write mixes and client concurrency levels?
- **RQ2:** How do throughput and latency change as the replica count varies, and how does the offered conflict rate affect throughput?
- **RQ3:** How do leader and replica failures affect the performance and availability of Raft and EPaxos?
- **RQ4:** How do the two approaches differ in resource utilisation, and how is that utilisation distributed across replicas?

We make the following contributions:

- **Reproducible benchmark infrastructure:** we build the Consensus Lab, which runs Raft and EPaxos in the same execution environment behind a common client harness and interface, providing a controlled basis for comparing their measured behaviour.
- **Controlled comparison of ordering structures:** we characterise throughput, latency, and per-replica resource utilisation of leader-based and leaderless consensus under read- and write-intensive workloads, varying conflict rates, client concurrency, and replica counts.
- **Failure behaviour analysis:** we measure the availability impact of leader and replica failures, including the cost of leader election in Raft and the effect of replica loss in EPaxos.
- **Open infrastructure:** the lab, its experiment configurations, and the analysis pipeline are released, so that the full measured dataset can be regenerated from the pinned upstream implementations and re-analysed.

To answer these questions, we conduct controlled experiments comparing Raft and EPaxos under comparable system configurations and workloads. The experiments measure throughput, latency, resource consumption, and the behaviour of the protocols under node failures. Because the experiments are conducted in a local environment rather than on physically distributed machines, the results should be interpreted as measurements of protocol behaviour under the specified experimental conditions rather than as universal measurements of performance in production-scale distributed systems.

The remainder of this paper is organised as follows. Section 2 provides background on distributed consensus and reviews leader-based (Paxos and Raft) and leaderless (EPaxos) approaches, together with related work. Section 3 describes the Consensus Lab, the implementations under test, the client harness and workload model, the metrics, and the experiment suite. Section 4 presents the experimental results on throughput and scalability, latency, resource utilisation, and failure recovery. Section 5 discusses the results with respect to the research questions and the cost of centralised leadership. Section 6 discusses threats to the validity of the measurements. Section 7 summarises the findings.

2 Background and Related Work

2.1 Distributed Consensus

Distributed consensus is the problem of enabling multiple distributed processes to agree on a common value or sequence of values despite failures and unreliable communication. In a replicated state machine, this agreement is particularly important because replicas must apply the same sequence of operations in order to maintain a consistent state. Without an agreement mechanism, different replicas may independently accept or apply different operations, causing their states to diverge.

Consensus therefore provides a fundamental abstraction for building fault-tolerant distributed systems. A consensus protocol must maintain safety, ensuring that correct replicas do not make contradictory decisions, while also providing liveness under the conditions assumed by the protocol. For replicated state machines, these properties are closely connected to the problem of establishing a consistent ordering of state-changing operations.

Consensus protocols can be broadly distinguished by how they coordinate this ordering. One major approach uses a distinguished leader that coordinates the replication of operations, while another distributes the coordination process among multiple replicas without relying on a single permanent ordering authority. Raft represents the former approach, while EPaxos represents the latter.

2.2 Leader-Based Consensus: From Paxos to Raft

Paxos is one of the foundational protocols for distributed consensus. It provides a formal approach to reaching agreement among distributed processes in the presence of failures. However, the original presentation of Paxos is highly abstract, and constructing a complete replicated state machine requires additional mechanisms and implementation decisions beyond the core consensus abstraction [2].

Raft was designed as an alternative consensus algorithm with understandability as a primary design goal. Rather than presenting consensus as a single abstract problem, Raft decomposes it into relatively independent components, including leader election, log replication, and safety [4]. This decomposition provides a more explicit structure for reasoning about how a replicated state machine reaches agreement and maintains correctness.

A central feature of Raft is its leader. At any given term, one replica acts as the leader while the remaining replicas act as followers. Client requests that modify replicated state are normally directed to the leader. The leader appends these operations to its log and replicates the corresponding entries to follower replicas. Once an entry has been replicated to a sufficient quorum, it can be committed and subsequently applied to the replicated state machines [4].

This architecture simplifies the ordering problem by establishing a clear ordering authority. Rather than requiring replicas to independently determine the ordering of every write, followers primarily replicate the log produced by the leader. The Raft design also defines safety properties and mechanisms for preserving them during log replication and changes in leadership [4].

However, the same structure introduces a dependency on the leader. Since the leader is the primary entry point for replicated writes, the write path is structurally concentrated around a single replica. Adding additional follower replicas therefore does not directly provide additional independent write-ordering capacity. Under sufficiently high write workloads, the leader may become a bottleneck.

The leader also introduces a failure mode that does not exist in the same form in a leaderless protocol. When the current leader becomes unavailable, the remaining replicas must elect a new leader before normal leader-based processing can resume. During this transition, new writes cannot be committed through the normal leader-based path. The election process therefore introduces a period in which failure of the ordering authority can affect request processing.

Thus, Raft provides a clear and structured approach to consensus by centralising the ordering of replicated operations around a leader. This design can simplify coordination and reasoning about correctness, while also creating dependence on the processing capacity and availability of the leader.

2.3 Leaderless Consensus: EPaxos

Leaderless consensus takes a different approach to distributed ordering. Instead of requiring all operations to pass through a single distinguished leader, multiple replicas can participate in proposing operations and determining their dependencies. EPaxos was designed around this approach, motivated in part by the limitations of leader-based consensus when a single replica becomes a central point for coordinating operations [3].

In EPaxos, a client can send a command to any replica, which can act as the replica responsible for proposing that command. This removes the requirement for a single replica to serve as the universal ordering point and allows operations to be proposed concurrently at different replicas [3].

Rather than imposing a single global ordering on all operations through a leader, EPaxos establishes dependencies between commands. Commands that are independent can potentially be committed without requiring them to be ordered relative to one another, while conflicting commands require their dependency relationships to be established consistently [3].

This design allows EPaxos to exploit command independence. When operations do not conflict, replicas can process proposals concurrently instead of serialising all operations through one leader. The resulting degree of parallelism depends on the workload and, in particular, on the extent to which concurrently proposed commands are independent [3].

Leaderless operation also changes the system’s behaviour under replica failures. The failure of a particular replica does not necessarily require the remaining replicas to elect a new global leader before they can continue proposing commands. As long as the protocol can obtain the required quorum and satisfy its communication assumptions, other replicas can continue participating in the consensus process [3].

However, removing the single ordering authority does not eliminate coordination. Because multiple replicas can propose commands concurrently, EPaxos must determine the dependencies between commands and ensure that replicas execute conflicting operations in a compatible order. These mechanisms introduce additional coordination and computation compared with a system in which a single leader directly determines the order of operations [3].

The resulting trade-off is therefore not simply between consistency and availability. Instead, leader-based and leaderless consensus place coordination costs in different parts of the system. Leader-based consensus concentrates ordering around a leader, while leaderless consensus distributes the responsibility for ordering across replicas and relies on dependency management to coordinate concurrent operations.

2.4 Related Work

Prior research has examined both the correctness and practical performance of consensus protocols. Paxos established a foundational framework for distributed agreement, while Raft provided a more explicitly structured leader-based approach with an emphasis on understandability [2, 4]. EPaxos subsequently explored a leaderless approach that exploits command independence and dependency relationships to reduce the reliance on a single ordering authority [3].

The original evaluations of Raft and EPaxos also demonstrate that consensus performance is influenced by system and workload characteristics. In particular, factors such as request distribution, command conflicts, communication conditions, and failures can affect the observed performance of different consensus architectures [3, 4].

However, the different ordering structures of leader-based and leaderless consensus imply different sources of coordination cost. In a leader-based protocol, the ordering process is concentrated around a distinguished replica, whereas a leaderless protocol distributes proposal and dependency coordination across multiple replicas.

We therefore experimentally compare Raft and EPaxos under controlled workloads and failure scenarios. We measure read and write performance, throughput, latency, resource consumption, and behaviour following replica failures. Rather than proposing a new consensus algorithm, we investigate how the different ordering architectures of Raft and EPaxos affect their observed performance and resource requirements.

3 Experimental Setup

3.1 Consensus Lab Architecture

The experiments are conducted with the Consensus Lab, a reproducible benchmark infrastructure that runs both protocols in the same execution environment. Each run is a Docker Compose project with one container per role: a coordination service (the upstream EPaxos master [3], or the lab’s raft master which mirrors its RPC surface), one container per replica, and a benchmark client. All containers share a single runtime image and identical CPU and memory allocations.

Both protocols expose the same benchmark-facing interface: the upstream EPaxos `generic smr` wire protocol (`PROPOSE` → `ProposeReplyTS`). A single client binary drives both protocols with the same workload generator, so the op mix, key distribution, and timing are identical. The only protocol-intrinsic difference is endpoint selection: Raft sends every request to the leader reported by HashiCorp Raft, while EPaxos spreads requests round-robin across replicas. Reads

are routed through consensus in both protocols; there is no Raft-only local-read optimisation.

3.2 Implementation

Raft is provided by `github.com/hashicorp/raft v1.7.3` [1], wrapped in a thin adapter (`lab/adapters/raft/`) that converts benchmark requests into state-machine commands, calls `raft.Raft.Apply`, waits for the resulting future, and returns the response. HashiCorp Raft performs leader election, log replication, quorum calculation, commit logic, term management, and state transitions; the adapter never elects or selects a leader. Raft replicas persist state to named volumes (boltdb).

EPaxos uses the upstream `efficient/epaxos` server and master unchanged [3] (pinned commit `791b115`, run with `-e -exec -dreply`). The lab only adds external orchestration: containers, monitoring, and failure injection. EPaxos replicas keep state in memory (upstream default). The vendored upstream hardcodes the dependency-set size to 5; the lab extends it to 9 via a documented wire-format extension so that 7- and 9-replica clusters run. The protocol’s phases, quorums, and decision rules are unchanged, and all replicas run the same patched binary.

3.3 Client Harness and Workload Model

The client generates requests with a configurable read/write mix over a keyspace of 1,000 keys. Write ratio is expressed as the percentage of requests that are writes; the remainder are reads. The client is closed-loop and batch-oriented: it issues requests up to a configured concurrency level and measures throughput and latency per batch. A warm-up phase precedes the measured phase, and warm-up requests are not recorded.

3.4 Metrics

For every request, the client records the run ID, protocol, replica count, workload ratio, concurrency, operation, key, target replica, start and end timestamps, latency, and success or failure. From these records the processing pipeline computes throughput (requests per second), success rate, and p50/p95/p99 latency distributions.

Per-replica resource samples are collected from the host at approximately 200 ms resolution: CPU from `cgroup v2 cpu.stat`, network RX/TX from `/proc/<pid>/net/dev` in the container’s network namespace, and memory (RSS) from `/proc/<pid>/status`. Raw samples are stored per replica without aggregation, so that workload distribution across replicas can be examined.

3.5 Testbed Configuration and Parameters

All runs use a 20-second measured phase with a 3-second warm-up and ten independent repetitions. Each replica container is allocated 2.0 CPUs and 512 MB of memory; the client gets 2.0 CPUs and 512 MB, and the master 1.0 CPU and 256 MB. `GOMAXPROCS` is set to 4 for every replica in both protocols. The request timeout is 2,000 ms. Raft uses a 1,000 ms

heartbeat timeout, a 2,000 ms election timeout, a snapshot threshold of 8,192 entries, and 1,024 trailing logs.

Each repetition is a fresh Docker Compose project: new containers, new networks, and new volumes. Raft’s persisted state (term, voted-for, log, snapshots, cluster membership) lives in per-replica volumes that are destroyed after every repetition and verified removed; persistence is not part of any experiment, so no consensus state survives between repetitions. EPaxos state is in-memory by construction. Before each repetition the harness verifies that the environment is clean (ports free, no leftover containers, volumes, or networks); after each repetition it verifies that teardown removed everything. A repetition that cannot be isolated is recorded as failed, never silently reused.

To prevent experimental conditions from being confounded with wall-clock time, conditions are not executed in blocks. Instead, the runner uses blocked randomization: repetition block b contains every configuration exactly once, in an order shuffled with a seeded random number generator. The schedule seed is recorded, so the exact execution order is reproducible. Each repetition uses an independent workload seed (base seed plus repetition index), recorded in the run’s metadata, so no two repetitions replay the same workload sequence and any anomalous run can be reproduced exactly.

The experiment dimensions are: replica count (3, 5, 7, 9), write ratio (0, 10, 50, 90, 100 %), conflict ratio (0, 10, 25, 50, 75, 90, 100 %), and client concurrency (1, 2, 4, 8, 16, 32, 64, 128, 256).

3.6 Experiment Suite

The experiment suite consists of six families, each with ten repetitions:

- **Workload:** write ratio sweep at 3 replicas and 8 concurrency levels (1–128), 2 protocols.
- **Scaling:** replica count sweep (3–9) at 100 % writes and concurrency 32, 2 protocols.
- **Conflict:** conflict ratio sweep (0–100 %) at 3 replicas, 100 % writes, and concurrency 32, 2 protocols.
- **Concurrency:** concurrency sweep (1–256) at 3 replicas and 100 % writes, 2 protocols.
- **Election:** Raft leader-election disruption: the leader is killed and the surviving replicas’ Raft transport is isolated for a target number of failed election attempts (0–5). The independent variable is the *measured* number of failed elections (derived from the Raft adapters’ dropped pre-vote and vote counters), not the configured target; the target is a nominal setting only. The measured phase is long enough for the longest isolation to expire and recovery to complete; a run whose cluster does not recover within the phase is recorded as failed.
- **Failure:** Raft leader kill, Raft follower kill, and EPaxos replica kill at 3 replicas, 100 % writes, and concurrency 32.

Failure injection is deterministic: the runner kills a container with `docker kill` at a configured offset into the measured phase. For Raft leader failure, the runner queries the raft master for the current leader, maps it to the container hostname, and kills it; the client observes the election and re-connects to the new leader. For EPaxos replica failure, replica 0 is killed and the client moves to the next replica. Recovery timing is reconstructed from request timestamps and the approximately 200 ms monitor resolution, and is therefore approximate.

Host-level telemetry (load average and available memory, sampled at 200 ms) is recorded for every run. Host suspension, scheduling gaps, CPU starvation, and clock jumps are detected and recorded per run. Anomalous runs are marked and reported; they are never silently deleted.

The exclusion rule is fixed in advance and applied uniformly to every condition: a run whose host telemetry was flagged contaminated is excluded from every aggregate, and the lost observation is replaced by re-running that repetition, so each condition keeps its full set of independent repetitions. A replacement is a separate run recording which attempt it replaces and why; the original attempt stays in the dataset. Excluded and replacement runs are reported, and the Execution Integrity section of the accompanying report states how many runs were affected.

All aggregates are computed from the raw per-run observations. Every reported value carries its sample size n ; means are reported with sample standard deviation and 95 % confidence intervals (Student’s t). Latency distributions are reported as p50/p95/p99. For the election experiment we report the Pearson correlation between the measured number of failed elections and the availability gap, with n and a t -based p -value, and we plot every observation. Outliers within clean runs are not removed.

4 Evaluation

All reported values are means over ten independent repetitions. Unless stated otherwise, results are for 3 replicas, 100 % writes, and concurrency 32. All figures are generated from the processed benchmark data by `lab/report/figures.py`; each figure shows the mean over repetitions as a line or bar, with the individual repetition values overlaid as points where repetitions are available.

4.1 Throughput and Scalability

Figure 1 shows the measured mean throughput at each tested replica count. Throughput decreased as the replica count increased for both protocols. At the 3-replica baseline (concurrency 32, 100 % writes), EPaxos recorded 5,330 req/s and Raft 3,531 req/s. From 5 to 9 replicas, EPaxos fell from 4,969 req/s to 2,283 req/s and Raft from 2,841 req/s to 1,768 req/s (Table 1). Relative to the 3-replica baseline, the 9-replica mean was 57% lower for EPaxos and 50% lower for Raft. The EPaxos/Raft ratio of the means was $1.51\times$ at 3 replicas, $1.75\times$ at 5, $1.36\times$ at 7, and $1.29\times$ at 9.

Table 1: Throughput (req/s, mean of 10 repetitions) by replica count, at 100 % writes and concurrency 32.

Replicas	Raft	EPaxos	EPaxos/Raft
3	3,531	5,330	$1.51\times$
5	2,841	4,969	$1.75\times$
7	2,108	2,864	$1.36\times$
9	1,768	2,283	$1.29\times$

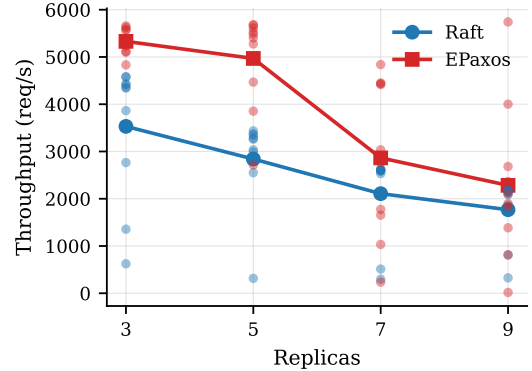


Figure 1: Mean throughput by replica count at 100 % writes and concurrency 32. Points show individual repetitions.

Because the same nominal configuration (3 replicas, 100 % writes, concurrency 32) is exercised independently by four experiment families, it also measures the run-to-run variability of a mean over ten repetitions. Those four means were 4,282 (workload), 4,006 (concurrency), 4,221 (conflict at 0 %) and 3,531 req/s (scaling) for Raft, and 4,926, 5,011, 5,147 and 5,330 req/s for EPaxos. EPaxos’s four means lie within 8% of each other; Raft’s span 21%, with the scaling-family mean the lowest and the most variable across repetitions (SD 1,453 req/s against a mean of 3,531). Protocol differences below this magnitude should not be read out of a single family.

Figure 2 shows the measured throughput across the concurrency sweep. Throughput increased with client concurrency for both protocols at every tested level. At concurrency 1, Raft recorded 257 req/s and EPaxos 174 req/s. At concurrency 256, EPaxos recorded 33,971 req/s and Raft 17,845 req/s. Across the sweep, the highest per-run value observed was 43,681 req/s for EPaxos and 21,656 req/s for Raft, both at concurrency 256. From concurrency 64 to 256, the Raft mean increased by a factor of 2.5 (7,020 to 17,845 req/s) and the EPaxos mean by a factor of 3.2 (10,712 to 33,971 req/s).

Figure 3 shows the measured throughput across the write-ratio sweep at concurrency 32. Between 0 % and 100 % writes, the mean for EPaxos ranged from 4,926 to 5,332 req/s and for Raft from 3,382 to 4,282 req/s. Neither protocol showed a

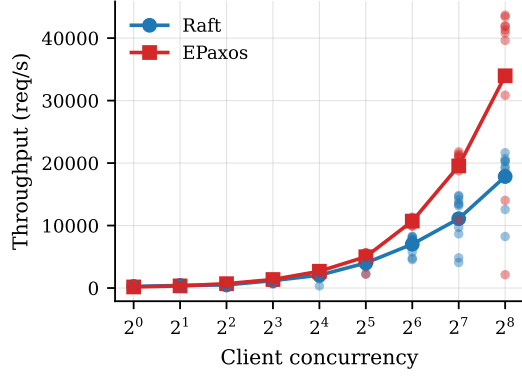


Figure 2: Mean throughput by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

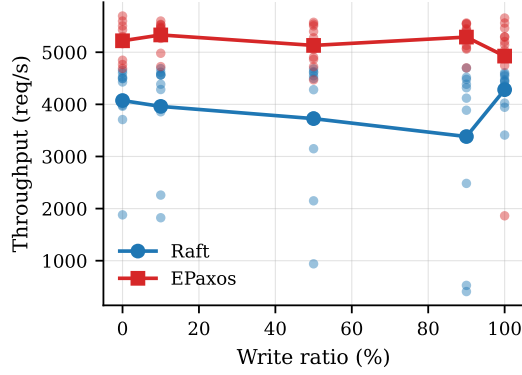


Figure 3: Mean throughput by write ratio at 3 replicas and concurrency 32. Points show individual repetitions.

systematic trend across the write ratios tested: Raft’s highest mean was at 100 % writes and its lowest at 90 %, while EPaxos’s highest was at 10 % writes (24% above its 100 % value).

Figure 4 shows the measured throughput across the conflict-ratio sweep. Mean throughput ranged from 3,514 to 4,425 req/s for Raft and from 4,662 to 5,235 req/s for EPaxos across conflict ratios of 0–100 %. The spread across repetitions within a configuration was of comparable magnitude to the spread across configurations in both cases.

4.2 Latency

Figure 5 shows the measured median (p50) latency at each tested replica count. At the 3-replica baseline, the median was 12.3ms for Raft and 5.6ms for EPaxos. As replicas were added, the Raft median rose to 16.8ms at 5 replicas and 25.5ms at 7, then fell to 22.8ms at 9, while the EPaxos

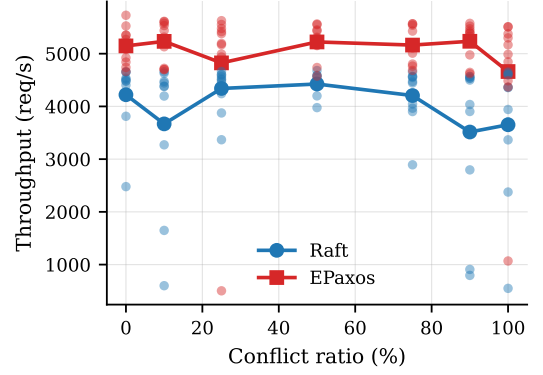


Figure 4: Mean throughput by conflict ratio at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions.

Table 2: Median (p50) latency in milliseconds by replica count, at 100 % writes and concurrency 32.

Replicas	Raft p50 (ms)	EPaxos p50 (ms)
3	12.3	5.6
5	16.8	5.6
7	25.5	6.0
9	22.8	6.2

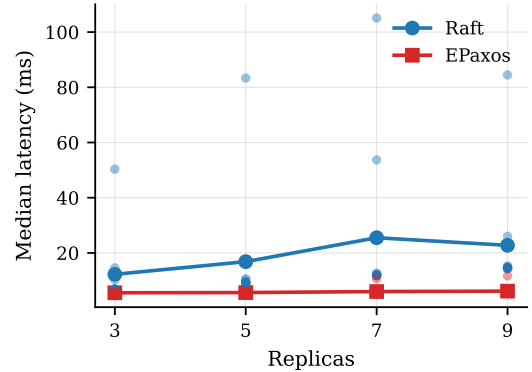


Figure 5: Median (p50) latency by replica count at 100 % writes and concurrency 32. Points show individual repetitions.

median stayed between 5.6 ms and 6.2 ms across the same range (Table 2).

Figure 6 shows the measured median latency across the concurrency sweep. Median latency increased with concurrency for both protocols. At concurrency 1, the Raft median was 4.1 ms and the EPaxos median 5.5 ms; at concurrency 32, 7.4 ms and 5.9 ms; and at concurrency 256, 13.4 ms and 14.0 ms.

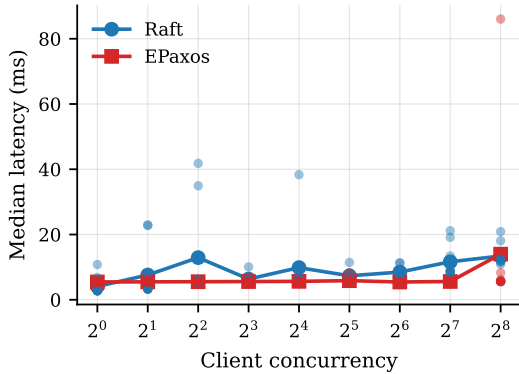


Figure 6: Median (p50) latency by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

Table 3: Failure behaviour at 3 replicas, 100 % writes, concurrency 32. Gap is the largest interval with no successful write completion after failure injection.

Failure	Protocol	Gap (s)	Failed req.
Leader kill	Raft	1.81–4.74	2,169–4,133
Follower kill	Raft	0.02–0.04	0
Replica kill	EPaxos	0.02–0.09	10–106

4.3 Resource Utilisation

Figure 7 summarises the per-replica resource measurements. Across all runs, mean per-replica CPU utilisation was 35.8% for Raft and 35.0% for EPaxos; mean peak per-replica RSS was 26.4 MB for Raft and 46.8 MB for EPaxos; and mean per-replica network rates were 441 kB/s RX and 443 kB/s TX for Raft, and 408 kB/s RX and 392 kB/s TX for EPaxos.

Figure 8 shows the per-replica CPU utilisation in the 9-replica scaling runs. In the Raft runs, the per-replica means ranged from 16.3% to 29.4%: whichever replica held the leader role during a repetition used more CPU, and the leadership rotated across repetitions, so each replica’s mean reflects the fraction of repetitions in which it led. In the corresponding EPaxos runs, the per-replica means ranged from 22.4% to 23.0%, with no replica standing out.

4.4 Failure Recovery Behaviour

Failure outcomes differed by failure type (Table 3, Figure 9). Killing the Raft leader produced a write-availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. Killing a Raft follower produced a gap of 0.02–0.04 s with no failed requests. Killing an EPaxos replica produced a gap of 0.02–0.09 s with 10–106 failed requests. All runs completed with status `success`; the failed requests are those issued during the transition.

The election family, which isolates replicas to force repeated failed elections during a Raft leader failure, produced

availability gaps that grow with the measured number of failed elections (Figure 10, left): across the 60 measured runs the Pearson correlation between the measured failed elections (0–13) and the gap is $r = 0.90$ ($p < 10^{-4}$). This correlation is partly mechanical: the injection isolates the surviving replicas’ transport for a duration proportional to the target, so a run with more failed elections has a longer gap by construction. To separate the injection duration from the recovery behaviour, the right panel shows the recovery time measured from the end of the isolation window to the first successful request. This decoupled metric is smaller and more variable but still increases with the measured election count ($r = 0.41$, $p = 0.001$, $n = 60$): after the isolation expires, a cluster that experienced more failed elections takes longer to restore service, beyond the extra time the injection itself consumed.

4.5 Summary of Findings

The measurements support the following observations under the conditions tested:

- At each tested replica count, the measured mean throughput for EPaxos was higher than that for Raft, with the largest relative difference at 5 replicas (1.75×); the 95% intervals are separated at 3 and 5 replicas but overlap at 7 and 9.
- Throughput decreased with replica count for both protocols; from 3 to 9 replicas the mean decreased by 57% for EPaxos and 50% for Raft.
- At concurrency 256, the measured mean throughput was 33,971 req/s for EPaxos and 17,845 req/s for Raft; the highest per-run values observed were 43,681 req/s and 21,656 req/s respectively.
- Median latency was lower for EPaxos at every tested replica count, and increased with replica count only for Raft.
- In the 9-replica runs, per-replica CPU means ranged from 16.3% to 29.4% in Raft, where leadership rotated between repetitions, and from 22.4% to 23.0% in EPaxos, where no replica stood out.
- Write ratio and conflict ratio produced no systematic throughput trend in either protocol over the ranges tested.
- Raft leader-kill experiments recorded write-availability gaps of 1.81–4.74 s and 2,169–4,133 failed requests, while follower-kill and EPaxos replica-kill experiments recorded gaps under 0.1 s with at most 106 failed requests.
- The measured failed-election count (0–13, never the configured target) predicts the availability gap almost linearly ($r = 0.90$, $n = 60$), and remains a significant predictor of the decoupled post-isolation recovery time ($r = 0.41$, $p = 0.001$).

5 Discussion

The experimental results show that Raft and EPaxos incur different costs as a consequence of their ordering structures.

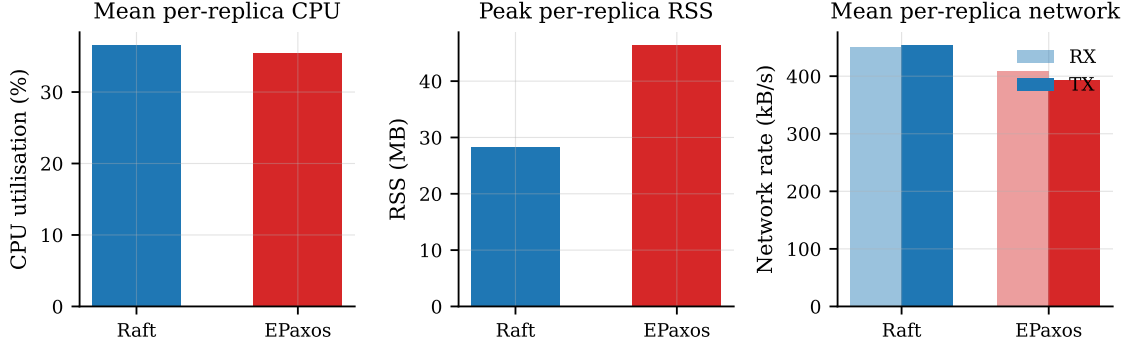


Figure 7: Mean per-replica CPU utilisation, peak per-replica RSS, and mean per-replica network rates, aggregated over all runs.

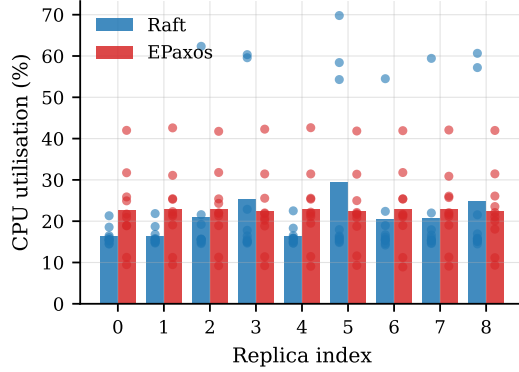


Figure 8: Per-replica CPU utilisation in the 9-replica scaling runs. Bars show the mean over repetitions; points show individual repetitions.

EPaxos generally achieved higher throughput in the evaluated workloads, while Raft remained competitive at low concurrency and used less memory. The results therefore indicate that the relative behaviour of the two protocols depends on workload intensity, replica scaling, and failure mode rather than on a single steady-state performance characteristic.

5.1 RQ1: Steady-State Performance

The throughput results show that EPaxos generally achieved higher throughput across the evaluated read/write configurations. Changing the read/write ratio did not produce a clear systematic change in the relative performance of the two protocols (Figure 3). Client concurrency was a more apparent factor in the observed performance difference (Figure 2). Median latency increased with concurrency for both protocols, from 4.1 ms at concurrency 1 to 13.4 ms at concurrency 256 for Raft, and from 5.5 ms to 14.0 ms for EPaxos. Thus, the latency measurements converged at the highest concurrency even though the throughput difference between the protocols became larger (Figure 6).

This behaviour is consistent with the different organisation of write processing in the two protocols. In Raft, committed writes are coordinated through the current leader, concentrating an important part of the write-processing path at a single replica. EPaxos does not require the same centralised ordering path, allowing a larger portion of consensus processing to be handled across multiple replicas. This distinction becomes more relevant as offered concurrency increases, because the Raft leader must continue to handle the central part of the replicated write path.

The results also show that the observed EPaxos advantage is not present at all concurrency levels. At concurrency 1 and 2, the mean throughput of Raft was higher than that of EPaxos, at 256.7 versus 173.9 req/s and 411.7 versus 339.4 req/s, respectively. The intervals were separated at concurrency 1 but overlapped at concurrency 2. At higher concurrency levels, the intervals were separated from concurrency 4 onwards except at concurrency 32, where they overlapped. Concurrency 32 was also the default configuration used in the scaling, conflict-ratio, and failure experiments.

When the offered workload is small, the Raft leader has sufficient capacity to handle the workload without the concentration of processing responsibility becoming an observable throughput bottleneck. Under these conditions, the additional coordination and protocol state associated with EPaxos can reduce or eliminate its throughput advantage. As concurrency increases, however, the centralised processing path becomes more relevant to the observed throughput difference.

These results therefore suggest that the cost associated with centralised leadership is workload-dependent. In the evaluated environment, it was more visible when the offered workload approached a level at which the processing capacity of the Raft leader became relevant, rather than appearing as a constant performance penalty across all concurrency levels.

5.2 RQ2: Scaling and Workload Sensitivity

Increasing the number of replicas produced different scaling behaviour in latency and throughput. EPaxos maintained comparatively stable latency as the replica count increased,

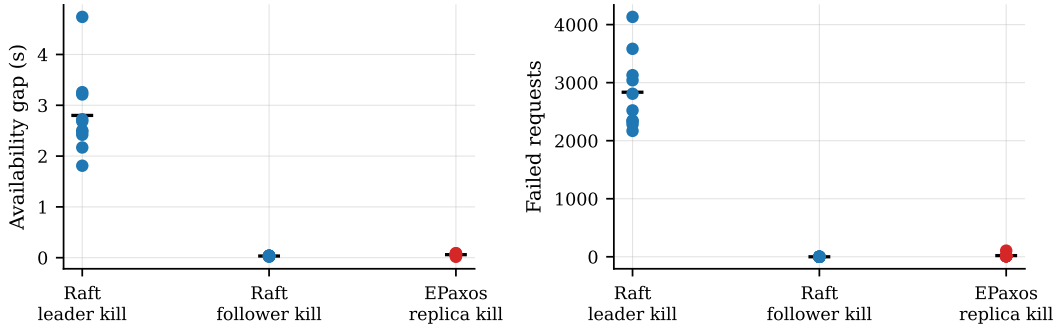


Figure 9: Write-availability gap and failed requests by failure type at 3 replicas, 100 % writes, concurrency 32. Points show individual repetitions; the bar marks the mean.

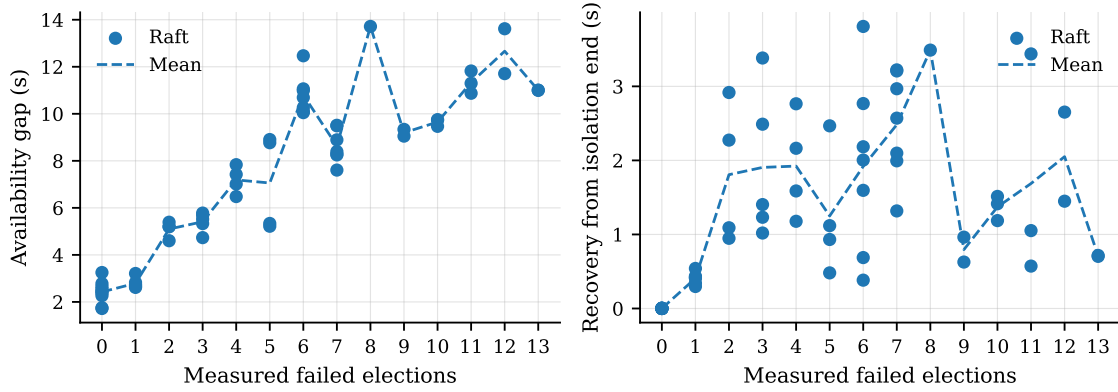


Figure 10: Election-failure recovery in Raft at 3 replicas, 100 % writes, concurrency 32. Left: write-availability gap vs the measured number of failed elections (the gap includes the isolation duration, which is proportional to the target). Right: recovery time from the end of the isolation window to the first successful request, decoupled from the injection duration. Points show individual repetitions; the dashed line marks the mean per value.

whereas Raft exhibited a more pronounced increase in latency (Table 2). EPaxos throughput nevertheless also decreased as additional replicas were introduced (Table 1). Thus, distributing ordering responsibility does not eliminate the communication and coordination costs associated with replication.

The throughput comparison should be interpreted together with the uncertainty intervals. At three and five replicas, the measured intervals for EPaxos and Raft were separated, whereas the intervals overlapped at seven and nine replicas. The results therefore provide clearer evidence of a throughput difference at the smaller replica counts, while the larger configurations require more cautious interpretation (Figure 1). The decrease in EPaxos throughput between five and seven replicas was large in terms of the sample means, but the associated variation also increased. It should therefore not be interpreted as a statistically distinct step without considering the reported uncertainty.

The scaling results consequently reveal different forms of cost rather than a single ordering of the two protocols. EPaxos incurs increasing coordination costs as the replica set grows, while Raft retains a central leader whose processing and communication responsibilities remain concentrated at one replica. The results do not indicate that either architecture eliminates the cost of increasing the replica count; instead, the measured effects appear in different performance dimensions.

The conflict-ratio experiment provides a further workload-sensitivity result. Across the evaluated conflict ratios, EPaxos throughput ranged from 4,662 to 5,235 req/s, while Raft throughput ranged from 3,514 to 4,425 req/s (Figure 4). The changes observed across conflict configurations were not clearly distinguishable from the variation between repetitions. Consequently, the experiment did not reveal a systematic throughput trend associated with the offered conflict ratio. Within the evaluated setup, the measurements therefore do not provide throughput-level evidence that changing the

conflict ratio produced a systematic change in the relative performance of the two protocols.

5.3 RQ3: Failure and Recovery

The failure experiments show that the effect of replica failure depends strongly on whether the failed replica is the Raft leader. Failures of Raft followers and EPaxos replicas produced only small availability gaps when sufficient replicas remained available to maintain progress. In contrast, direct Raft leader-failure experiments produced a median gap of 2.59 seconds, with observed values ranging from 1.81 to 4.74 seconds (Table 3, Figure 9).

The larger disruption following a Raft leader failure is consistent with the recovery steps required by the leader-based architecture. After detecting the leader failure, the remaining replicas must proceed through timeout and leader election before a new leader can be established and normal leader-based processing can resume. This creates a period during which requests cannot be processed through the normal leader-based path.

The difference is also visible at the request level. Raft leader-failure experiments produced approximately 2,169–4,133 failed requests, whereas the corresponding EPaxos replica-failure experiments produced approximately 10–106 failed requests. The measured difference therefore involved not only increased latency but also a substantially larger interruption in successfully processed requests under the evaluated failure conditions.

The separate election-injection experiments require a different interpretation (Figure 10). The observed isolation gap reached 13.72 seconds, but this value should not be treated as the recovery time of Raft. The experimental procedure intentionally isolates the target for a duration related to the injected election condition, meaning that part of the measured gap is determined by the experimental procedure itself. When recovery is measured from the end of the isolation period, the median recovery time was 1.15 seconds and the maximum was 3.81 seconds. The decoupled recovery measure showed a correlation of $r = 0.41$ ($p = 0.001$, $n = 60$).

The failure results therefore identify leader failure as a distinct recovery cost in the evaluated Raft configuration. The much longer gaps observed in the election-injection experiment should instead be interpreted as a combination of the injected isolation period and subsequent recovery behaviour, rather than as direct measurements of normal leader-failure recovery time.

5.4 RQ4: Resource Utilisation and Load Distribution

The resource measurements reveal another difference between the two architectures. Overall CPU utilisation was broadly similar between Raft and EPaxos, indicating that the observed throughput difference cannot be explained simply by one protocol consuming substantially more total CPU resources.

The distribution of CPU utilisation across individual replicas was different, however. Individual observations ranged from approximately 14.2–69.8% for Raft and 8.9–42.6% for EPaxos (Figure 8). These observations are consistent with Raft concentrating a greater portion of processing at the current leader, while EPaxos distributes processing more evenly across replicas. This distinction is particularly relevant because averaging across repeated runs can obscure processing concentration when leadership changes between replicas.

The difference in resource distribution provides another observable consequence of the different ordering structures. In Raft, the leader is responsible for a central part of the consensus path, so its resource utilisation can become substantially higher than that of other replicas. EPaxos instead distributes more of the processing responsibility across the replica set. The cost associated with centralised leadership can therefore appear not only in aggregate throughput but also in the distribution of computational load across replicas.

EPaxos, however, incurred a higher memory cost in the evaluated implementation. Its peak memory usage was approximately 1.8 times that of Raft (Figure 7). A possible explanation is that EPaxos maintains additional protocol state and metadata associated with distributed ordering and dependency management. The experiment does not isolate the individual sources of memory consumption, however, so the result should be interpreted as an observed resource-level trade-off rather than evidence that a particular protocol component is solely responsible for the difference.

5.5 Overall Discussion

Taken together, the results characterise Raft and EPaxos as making different engineering trade-offs rather than establishing one protocol as uniformly superior. EPaxos generally provided higher throughput in the evaluated workloads and maintained comparatively stable latency as the replica count increased. Its processing load was also distributed more evenly across replicas. These observations were accompanied by higher peak memory consumption and a reduction in throughput as the replica set became larger.

Raft, in contrast, remained competitive at low concurrency, where its leader had sufficient capacity to handle the offered workload. Its lower memory consumption also represents a practical resource advantage in the evaluated configuration. At higher concurrency, however, its centralised processing structure was associated with a larger throughput difference relative to EPaxos. Raft also exhibited a distinct recovery path when the current leader failed.

The failure results are particularly relevant to the cost of centralised leadership. A follower failure does not necessarily interrupt Raft’s progress while a sufficient quorum remains available, but failure of the leader changes the role responsible for the normal processing path. The system must establish a replacement leader before normal leader-based processing can continue. This produces a failure mode that differs from the failure of a non-leader replica.

The results therefore identify two observable effects associated with centralised leadership in the evaluated Raft implementation. During normal operation, the leader can become a concentration point for request processing as workload increases. During failure, the same central role creates a dependency on establishing a replacement leader before normal processing can resume. EPaxos avoids this particular centralised dependency, but does so with its own measured costs in memory consumption, coordination, and replica scaling.

Overall, the experiments do not establish that EPaxos is universally preferable to Raft. Rather, they show that the architectural choice changes where the costs of consensus appear: Raft concentrates processing and recovery responsibility around the leader, whereas EPaxos distributes more responsibility across replicas while incurring greater protocol-state and coordination costs. The interpretation of the measured performance therefore depends on the conditions under which the systems are compared rather than on a single performance ordering.

The results should also be interpreted within the limitations of the experimental setup. The client is closed-loop, so measurements at the highest concurrency may include client-side limitations that constrain both protocols and compress the observed difference. Measuring the same nominal configuration in four experiment families produced means that differed by up to 21% for Raft and 8% for EPaxos, so differences smaller than this reproducibility band should not be inferred from a single experiment family. Section 6 discusses these and the remaining threats to validity.

6 Threats to Validity

We distinguish four classes of threat.

6.1 Construct Validity

Throughput is the number of successful requests in the measured phase divided by that phase’s duration, and latency is measured by the client around each request. Both therefore include client-side queueing and the full consensus round trip; they are not server-side service times. The client is closed-loop and batch-oriented, so the reported throughput at a given concurrency is the rate the system sustains under closed-loop demand of that intensity, not the rate it would sustain under an open-loop load with the same number of outstanding requests. At the highest concurrency (Figure 2) the client and the container scheduler may themselves limit the observed throughput, which would compress rather than inflate the difference between the protocols.

Resource metrics are read from the host at approximately 200 ms resolution, from `cgroup v2 cpu.stat`, the container’s network namespace, and `/proc`. Cumulative counters are differenced during processing, so a sample is an average over an interval rather than a peak, and memory is RSS, which excludes page cache and memory the runtime has returned to the operating system.

The conflict rate is realised by routing a configured fraction of requests to shared hot keys, and the realised fraction is recorded per request, so the measured rate is not assumed. The reported conflict result is nevertheless a throughput-level observation: the EPaxos fast- and slow-path counters are recorded per run but are not part of the aggregated analysis.

6.2 Internal Validity

Each repetition is a fresh Docker Compose project with new containers, networks, and volumes, verified before and after the run, so no consensus state carries over between repetitions. Conditions are interleaved in reproducibly seeded random blocks rather than executed in condition blocks, and every repetition uses an independent workload seed, which removes the condition-order confounding and the repeated-workload-sequence effects present in the earlier version of the protocol.

Isolation is enforced at the container level but not at the host level: the benchmark itself occupies roughly nine of the measurement host’s twelve cores, so replicas contend with each other and with the client for CPU. We detect rather than prevent this. Host telemetry is sampled for every run, runs whose telemetry shows a suspension, a clock jump, or CPU starvation are excluded from every aggregate and re-run, and the replacement is recorded alongside the original attempt. Twelve of 1,302 measured runs were excluded this way. The same mechanism cannot rule out interference that leaves no telemetry signature.

The two systems are not equally mature. HashiCorp Raft is a widely deployed library, whereas the EPaxos implementation is the research prototype published with the original paper, run with its default `-e -exec -dreply` configuration. Part of any measured difference may therefore reflect implementation quality and tuning rather than the ordering architecture. One deviation is our own: the vendored EPaxos hardcodes a dependency-set size of five, which we extended to nine through a documented wire-format change so that 7- and 9-replica clusters can run at all. The protocol’s phases, quorums, and decision rules are unchanged, but the extension is exercised precisely in the configurations reported in Table 1.

Neither protocol is measured in a restart-from-persistence scenario. Failure injection kills a container and does not restart it, and Raft’s persisted state is destroyed between repetitions because persistence is not part of any experiment. EPaxos keeps its state in memory by upstream default, so a lost replica cannot rejoin with its prior state in either case. Failure timing is driven at a fixed offset into the measured phase and the availability gap is reconstructed from request timestamps at approximately 200 ms resolution, so recovery durations are approximate and batch-granular rather than a precise availability measurement. In the election experiment the isolation duration is a property of the injection rather than of the protocol, which is why the availability gap correlates with the measured election count partly by construction, and

why we also report recovery measured from the end of the isolation window (Section 4).

6.3 External Validity

All replicas run on a single host inside containers over a bridge network, so the measurements contain no wide-area latency, no real packet loss, and no multi-machine failure domains. The host is a WSL2 environment, which places a virtualisation layer between the containers and the hardware. Cluster sizes are 3–9, the key space is 1,000 keys, commands are uniform in size, and the measured phase is 20 s. The results therefore describe short, uniform, single-host runs rather than long-lived deployments with skewed access patterns or large payloads.

Reads are routed through consensus in both protocols and no Raft-specific read optimisation is enabled, so the read comparison is one of equivalent consistency rather than of each system’s fastest read path; a deployment using leader leases or `ReadIndex` would show a different read profile. Both systems are measured with identical CPU and memory allocations, which is the fair choice for a controlled comparison but is not a tuned deployment of either.

6.4 Statistical Conclusion Validity

Each configuration is measured ten times, and every aggregate is reported with its sample size, standard deviation, and a t -based 95 % interval. Ten repetitions leave wide intervals for a variable system: measuring the same nominal configuration in four experiment families produced means that differed by up to 21 % for Raft and 8 % for EPaxos, and intervals overlap in several comparisons whose means differ, for example at 7 and 9 replicas. We therefore read only differences that exceed this reproducibility band and treat the overlapping comparisons as inconclusive rather than as small effects. Numerous configurations are compared without a multiplicity correction, so an isolated configuration-level difference should not be read as an established effect. Outliers within clean runs are not removed.

7 Conclusion

This study experimentally compared Raft, a leader-based consensus protocol [4], with EPaxos, a leaderless protocol [3]. Under the evaluated workloads and experimental conditions, Raft provided a comparatively simple and stable coordination structure, while EPaxos achieved higher throughput and lower latency in several of the evaluated scenarios and showed greater tolerance to individual replica failures.

The results also highlight the costs associated with centralised leadership. In particular, Raft exhibited a noticeable availability cost when the leader failed, as well as limitations in write scalability as the number of replicas increased. These observations suggest that the leader can serve as an effective point for simplifying coordination and maintaining a clear ordering structure, but can also become a bottleneck for write processing and a single point of disruption during failures. In contrast, EPaxos distributes coordination across replicas,

reducing its dependence on a single leader at the cost of a more complex consensus mechanism.

These findings should be interpreted within the scope of the experimental setup. The compared systems use different implementations, and implementation-level differences may affect the measured performance: Raft is represented by the widely deployed HashiCorp Raft library [1], whereas the EPaxos measurements use the research prototype published with the original protocol [3]. Furthermore, the experiments were conducted in a controlled local environment rather than a geographically distributed production deployment. Therefore, the measured performance and failure behaviour should not be interpreted as universal characteristics of the protocols in all deployment environments (Section 6).

Overall, the results demonstrate a fundamental trade-off between centralised and decentralised coordination in distributed consensus: leader-based coordination can simplify system behaviour and reasoning, while leaderless coordination can reduce dependence on a single coordination point and provide advantages under certain workloads and failure conditions.

References

- [1] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [2] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [3] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [4] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>